

1. Register Move Instructions

Move instructions transfer 32-bit values between registers. ARM provides two variants: MOV (plain move) and MVN (Move Not, which moves the bitwise complement).

1.1 MOV and MVN Summary Table

Mnemonic	Operation	Description
MOV r0, r2	$r0 := r2$	Move 32-bit value from r2 into r0
MVN r0, r2	$r0 := \sim r2$	Move the bitwise complement of r2 into r0

1.2 Examples

Instruction	Effect	Notes
MOVS r2, r3	$r2 := r3$	Transfers data from r3 to r2; S suffix updates CPSR flags
MVNEQ r1, r2	$r1 := \sim r2$ (if Z flag set)	Negates r2 value and transfers to r1; executes only if Zero flag is set (EQ condition)

1.3 Instruction Detail: MOVS R2, R3

The MOVS instruction moves the value from one register to another and updates the condition flags in the CPSR (Current Program Status Register).

- R2 = Destination register (stores the value)
- R3 = Source register (value to be moved)
- S suffix (Set condition flags) = Updates CPSR flags based on the value moved

```
MOVS R2, R3 ; R2 = R3, and updates condition flags
```

1.4 Instruction Detail: MVNEQ R1, R2

MVN (Move Not) performs a bitwise NOT on the operand and stores the result in the destination register. The EQ condition means the instruction executes only if the Zero (Z) flag is set (i.e., the previous result was zero).

- MVN (Move Not) = Performs bitwise NOT on R3 ($\sim R3$)
- EQ (Equal Condition) = Executes only if the Zero (Z) flag is set
- R2 = Destination register (stores $\sim R3$)
- R3 = Source register (value to be inverted)

```
MVNEQ R2, R3    ; If Z == 1, R2 = NOT(R3)
```

2. Compare Instructions

Compare instructions perform arithmetic or logical operations and set the CPSR condition flags, but do NOT store the result in a register. They are used to drive conditional branches and conditional execution.

Mnemonic	Operation	Description
CMP r1, r2	$r1 - r2$	Compare two 32-bit values; sets flags based on subtraction result
CMN r1, r2	$r1 - (-r2)$	Compare negated; effectively adds $r1 + r2$ and sets flags
TST r1, r2	$r0 := r1 \& r2$	Test bits of 32-bit values using bitwise AND; sets flags
TEQ r1, r2	$r0 := r1 \wedge r2$	Test for equality of two 32-bit values using XOR; sets flags

Note: All compare instructions always update the CPSR flags (they implicitly behave like the 'S' suffix variant). No destination register is written.

3. Immediate Operands

Immediate addressing mode puts a raw constant as one of the operands, replacing the second source operand with a literal constant value preceded by '#'. This allows constants to be encoded directly in the instruction.

3.1 Basic Examples

```
ADD r3, r3, #1      ; r3 := r3 + 1
AND r8, r7, #&FF    ; r8 := r7[7:0]    (&: hexadecimal prefix)
```

3.2 Valid Immediate Value Ranges

Not every 32-bit value can be an immediate. ARM encodes immediates as an 8-bit value rotated right by an even number of positions. The valid ranges are:

Decimal Range	Hex Range	How it's encoded
0 – 255	0x00 – 0xFF	Direct 8-bit value, no rotation
256, 260, 264, ..., 1020	0x100 – 0x3FC, step 4	0x40–0xFF ror 30
1024, 1040, 1056, ..., 4080	0x400 – 0xFF0, step 16	0x40–0xFF ror 28
4096, 4160, 4224, ..., 16320	0x1000 – 0x3FC0, step 64	0x40–0xFF ror 26

3.3 Loading a Specific Immediate

The assembler automatically converts decimal or hex constants into the internal rotate format:

```
MOV r0, #0x40, 26    ; => MOV r0, #0x1000    (i.e. 4096)
MOV r0, #4096        ; => MOV r0, #0x1000    (i.e. 0x40 ror 26)
```

3.4 Bitwise Complement via MVN

Bitwise complements can also be formed using MVN. For example:

```
MOV r0, #0xFFFFFFFF ; assembles to MVN r0, #0
```

If the required constant cannot be generated by any rotation, the assembler reports an error.

4. Shifted Register Operands

ARM's second source operand can optionally be passed through the Barrel Shifter before being used by the ALU. This is a powerful feature unique to ARM that allows shifts and rotations to be performed for free alongside another operation.

4.1 Shift Types Overview

Mnemonic	Full Name	Shift Notation	Result
LSL	Logical Shift Left	$x \text{ LSL } y$	$x \ll y$
LSR	Logical Shift Right	$x \text{ LSR } y$	$(\text{unsigned}) \ x \gg y$
ASR	Arithmetic Shift Right	$x \text{ ASR } y$	$(\text{signed}) \ x \gg y$
ROR	Rotate Right	$x \text{ ROR } y$	$((\text{unsigned})x \gg y) \mid (x \ll (32-y))$
RRX	Rotate Right Extended	$x \text{ RRX}$	$(C \text{ flag} \ll 31) \mid ((\text{unsigned})x \gg 1)$

4.2 Logical Shift Left (LSL)

Bits are shifted left by y positions. Zeros fill from the right. The most significant bit(s) shifted out go into the Carry (C) flag.

```
MOV R0, R2, LSL #2 ; R0 := R2 << 2 (R2 unchanged)
```

Example: If $R2 = 0x00000030$ (binary: $0\dots0 \ 0011 \ 0000$)

- Before: $R2 = 0x00000030$
- After: $R0 = 0x000000C0$ (shifted left 2 bits = multiply by 4)
- $R2$ remains unchanged

4.3 Logical Shift Right (LSR)

Bits are shifted right by y positions. Zeros fill from the left (unsigned behavior). The least significant bit(s) shifted out go into the Carry flag.

```
MOV R0, R2, LSR #2 ; R0 := R2 >> 2 (R2 unchanged, unsigned)
```

Example: If $R2 = 0x00000030$ (binary: $0\dots0 \ 0011 \ 0000$)

- Before: $R2 = 0x00000030$
- After: $R0 = 0x0000000C$ (shifted right 2 bits = divide by 4, unsigned)
- $R2$ remains unchanged

4.4 Arithmetic Shift Right (ASR)

Like LSR but the MSB (sign bit) is replicated, preserving the sign of a signed integer. This is equivalent to signed division by a power of 2.

```
MOV R0, R2, ASR #2 ; R0 := R2 >> 2 (signed, R2 unchanged)
```

Example: If R2 = 0xA0000030 (MSB = 1, so negative number)

- Before: R2 = 0xA0000030
- After: R0 = 0xE800000C (MSB replicated; top bits become 1s)
- Positive operand: zeros fill from left (same as LSR)
- Negative operand: ones fill from left (sign preserved)

4.5 Rotate Right (ROR)

Bits shifted out from the right wrap around and enter from the left. No bits are lost. This is a circular rotation.

```
MOV R0, R2, ROR #2 ; R0 := R2 rotate right by 2 (R2 unchanged)
```

Example: If R2 = 0x00000031 (binary: 0...0 0011 0001)

- Before: R2 = 0x00000031
- After: R0 = 0x4000000C (lower 2 bits wrap to upper 2 bits)

4.6 Rotate Right Extended (RRX)

A 33-bit rotation using the Carry (C) flag as an extra bit. The register is shifted right by 1, the old C flag enters at the MSB (bit 31), and bit 0 becomes the new C flag.

```
MOV R0, R2, RRX ; R0 := (C_flag << 31) | (R2 >> 1), updates C
```

Example: If R2 = 0x00000031, C = 1

- Before: R2 = 0x00000031, C = 1
- After: R0 = 0x80000018, C = 1 (old C=1 moved to bit 31; bit 0 becomes new C)
- R2 remains unchanged

4.7 Shift Count from Register

Instead of a fixed immediate, the shift amount can come from a register. Only the bottom 8 bits of that register are used as the shift count. This is useful for dynamic array indexing.

```
ADD R0, R1, R2, LSL R3 ; R0 := R1 + R2 * 2^R3 (array index calc)
```

4.8 Fast Multiplication Using Shifted Operands

Multiplication by constants that are powers of 2 plus or minus 1 can be done in a single cycle using shifts. Example: multiply R0 by 35:

Method	Code	Explanation
--------	------	-------------

Using MUL	MOV R1, #35 MUL R2, R0, R1	Standard multiply instruction
Using shifts(faster)	ADD R0, R0, R0, LSL #2 R2, R0, R0, LSL #3	Step 1: $R0' = R0 + R0*4 = 5*R0$ Step 2: $R2 = 8*R0' - R0' = 7*R0' = 35*R0$

4.9 Shift Syntax Reference

Shift Operation	Syntax
Immediate	#immediate
Register	Rm
Logical shift left by immediate	Rm, LSL #shift_imm
Logical shift left by register	Rm, LSL Rs
Logical shift right by immediate	Rm, LSR #shift_imm
Logical shift right by register	Rm, LSR Rs
Arithmetic shift right by immediate	Rm, ASR #shift_imm
Arithmetic shift right by register	Rm, ASR Rs
Rotate right by immediate	Rm, ROR #shift_imm
Rotate right by register	Rm, ROR Rs
Rotate right with extend	Rm, RRX

5. Barrel Shifter

The Barrel Shifter is a hardware component inside the ARM processor that sits between the register file and the ALU. It pre-processes Operand 2 before it enters the ALU.

5.1 How It Works

In every ARM data processing instruction, the second operand passes through the Barrel Shifter. The ALU receives Operand 1 directly and Operand 2 after optional shifting/rotation.

- Operand 1 goes directly to the ALU
- Operand 2 goes through the Barrel Shifter first, then to the ALU
- The result comes out of the ALU

The Barrel Shifter can shift 32 bits right or left by a specific number of positions. The shift value can be:

- A 5-bit unsigned immediate (0-31 positions)
- Specified in the bottom byte of another register

5.2 Relationship to Immediate Operands

For immediate operands, the Barrel Shifter is used to encode a wider range of constants. An 8-bit number (range 0-255) is rotated right through an even number of positions to represent the final 32-bit constant. This preprocessing allows an increased range of 32-bit constants to be loaded directly into registers.

5.3 Use for Multiplication by Constants

The Barrel Shifter enables single-cycle multiplication by constants that are (power of 2) +/- 1, without needing the MUL instruction. Multiplication by general constants can be broken into multiple shift-and-add instructions.

Mnemonic	Operation	Examples
ADD r0, r1, r1, LSL #2	$r0 := r1 + r1 * 4$	$r0 = r1 * 5$ ($r0 = r1 + r1 * 4$)
ADD r2, r3, r3, LSL #4	$r2 := r3 * 17$	$r2 = r3 * 17$ ($r2 = r3 + r3 * 16$)
RSB r2, r2, r2, LSL #3	$r2 := r2 * 7$	$r2 = r2 * (8 - 1) = 7 * r2$

6. Multiply Instructions

The multiply instruction multiplies a pair of registers (Rm, Rs) and optionally accumulates the product with a third register (Rn for MLA). The result is stored in a register or a pair of registers.

6.1 Syntax

```
MUL{<cond>}{S} Rd, Rm, Rs          ; Rd := (Rm * Rs) [31:0]
MLA{<cond>}{S} Rd, Rm, Rs, Rn       ; Rd := ((Rm * Rs) + Rn) [31:0]
```

6.2 Basic Examples

Instruction	Effect
MUL r4, r3, r2	r4 := (r3 * r2) [31:0]
MLA r4, r3, r2, r1	r4 := ((r3 * r2) + r1) [31:0]

6.3 All Multiply Variants

Opcode [23:21]	Mnemonic	Meaning	Effect
000	MUL	Multiply (32-bit result)	Rd := (Rm * Rs) [31:0]
001	MLA	Multiply-accumulate (32-bit)	Rd := (Rm * Rs + Rn) [31:0]
100	UMULL	Unsigned multiply long	RdHi:RdLo := Rm * Rs
101	UMLAL	Unsigned multiply-accumulate long	RdHi:RdLo += Rm * Rs
110	SMULL	Signed multiply long	RdHi:RdLo := Rm * Rs
111	SMLAL	Signed multiply-accumulate long	RdHi:RdLo += Rm * Rs

6.4 Register Restrictions (Booth's Algorithm)

ARM uses Booth's algorithm for integer multiplication, which looks at transitions (1-to-0 and 0-to-1) in the binary pattern of the multiplier. The ARM7TDMI MUL executes in minimum 2 clock cycles and maximum 5 clock cycles. There are two key restrictions:

- Rd and Rm cannot be the same register. This can be avoided by swapping Rm and Rs (multiplication is commutative).
- PC (R15) cannot be used in a multiply instruction.

6.5 Multiplication by Constants

Multiplication by a constant equal to (power of 2) +/- 1 can be done in a single cycle using MOV, ADD or RSB with an inline shift:

```
MOV R1, #35
MUL R2, R0, R1      ; Standard: R2 = 35 * R0

; Or faster using shifts:
ADD R0, R0, R0, LSL #2  ; R0' = 5 * R0
RSB R2, R0, R0, LSL #3  ; R2 = 7 * R0' = 35 * R0
```

7. Flow Control Instructions

Flow control instructions determine which instruction executes next. ARM provides branch instructions that can transfer control to any address within a 32MB range (using a PC-relative offset).

7.1 Branch Instruction Types

Mnemonic	Full Name	Effect
B	Branch	pc = label (PC-relative offset, within 32MB)
BL	Branch with Link	pc = label; lr = address of next instruction after BL
BX	Branch Exchange	pc = Rm & 0xFFFFFFFF, T = Rm & 1
BLX	Branch Exchange with Link	pc = label, T = 1 OR pc = Rm & 0xFFFFFFFF, T = Rm & 1; lr = address of next instruction after BLX

7.2 Syntax

```
B{<cond>} label
BL{<cond>} label
BX{<cond>} Rm
BLX{<cond>} label | Rm
```

7.3 Simple Branch Example

```
        B label          ; unconditional jump to label
        ...
label:   ...              ; execution continues here
```

7.4 Conditional Branch / Loop Example

```
loop:    MOV R0, #0
        ...
        ADD R0, R0, #1
        CMP R0, #10
        BNE loop          ; if R0 != 10, go back to loop
```

7.5 Branch and Link (Function Calls)

BL saves the return address into R14 (the Link Register, LR). To return from the subroutine, copy LR back into the PC.

```
BL    sub           ; call sub; LR = address of next instruction
CMP   R1, #5        ; execution returns here after sub
MOVEQ R1, #0
...
sub:  ...           ; subroutine entry point
...
MOV   PC, LR        ; return: PC = LR
```

8. Branch Conditions

ARM supports 16 condition codes that can be appended to almost any instruction. Conditions are tested against the CPSR flags: N (Negative), Z (Zero), C (Carry), V (Overflow).

Mnemonic	Condition Name	CPSR Flags Required	Typical Use
EQ	Equal	$Z = 1$	Comparison equal or zero result
NE	Not Equal	$Z = 0$	Comparison not equal / non-zero result
CS / HS	Carry Set / Unsigned Higher or Same	$C = 1$	Arithmetic carry-out; unsigned higher or same
CC / LO	Carry Clear / Unsigned Lower	$C = 0$	No carry; unsigned lower
MI	Minus / Negative	$N = 1$	Result is negative
PL	Plus / Positive or Zero	$N = 0$	Result is positive or zero
VS	Overflow Set	$V = 1$	Signed overflow occurred
VC	Overflow Clear	$V = 0$	No signed overflow
HI	Unsigned Higher	$C=1, Z=0$	Unsigned comparison gave higher
LS	Unsigned Lower or Same	$Z=1$ or $C=0$	Unsigned comparison gave lower or same
GE	Signed Greater Than or Equal	$N=V$	Signed comparison $\geq$
LT	Signed Less Than	$N \neq V$	Signed comparison $<$
GT	Signed Greater Than	$Z=0, N=V$	Signed comparison $>$
LE	Signed Less Than or Equal	$Z=1$ or $N \neq V$	Signed comparison $\leq$
AL	Always (Unconditional)	ignored	Always executes (default)

9. Conditional Execution

A key ARM feature: ANY instruction can be made conditional by appending a condition code. This avoids short branches and keeps the pipeline flowing. The rule of thumb: if a conditional sequence is 3 instructions or fewer, conditional execution is better than using a branch.

9.1 Example: `if (R0 != 5) { R1 = R1 + R0 - R2 }`

Traditional approach using a branch:

```
CMP    R0, #5
BEQ    bypass      ; if R0 == 5, skip the block
ADD    R1, R1, R0   ; R1 = R1 + R0
SUB    R1, R1, R2   ; R1 = R1 - R2
bypass: ...
```

Better approach using conditional execution (smaller and faster):

```
CMP    R0, #5
ADDNE  R1, R1, R0   ; executes only if R0 != 5
SUBNE  R1, R1, R2   ; executes only if R0 != 5
```

9.2 Example: `if ((R0==R1) && (R2==R3)) R4++`

Traditional approach:

```
CMP    R0, R1
BNE    skip
CMP    R2, R3
BNE    skip
ADD    R4, R4, #1
skip: ...
```

Optimized using conditional execution (3 instructions vs 6):

```
CMP    R0, R1      ; sets flags for R0 vs R1
CMPEQ  R2, R3      ; only compare R2 vs R3 if R0==R1
ADDEQ  R4, R4, #1   ; only increment if both conditions hold
```

10. Single Register Load / Store

ARM is a load-store architecture: data must be loaded from memory into registers before processing, and stored back to memory when done. Data items can be 8-bit bytes, 16-bit halfwords, or 32-bit words. Addresses must be boundary aligned (e.g., word addresses must be multiples of 4).

10.1 Basic Load/Store Instructions

```
Syntax: <LDR|STR>{<cond>}{B} Rd, addressing
        LDR{<cond>}SB|H|SH Rd, addressing
        STR{<cond>}H Rd, addressing
```

Mnemonic	Operation	Effect
LDR	Load word into a register	$Rd \leftarrow \text{mem32}[\text{address}]$
STR	Save word from a register	$Rd \rightarrow \text{mem32}[\text{address}]$
LDRB	Load byte into a register	$Rd \leftarrow \text{mem8}[\text{address}]$
STRB	Save byte from a register	$Rd \rightarrow \text{mem8}[\text{address}]$
LDRH	Load halfword into a register	$Rd \leftarrow \text{mem16}[\text{address}]$
STRH	Save halfword from a register	$Rd \rightarrow \text{mem16}[\text{address}]$
LDRSB	Load signed byte into a register	$Rd \leftarrow \text{SignExtend}(\text{mem8}[\text{address}])$
LDRSH	Load signed halfword into a register	$Rd \leftarrow \text{SignExtend}(\text{mem16}[\text{address}])$

Note: There is no STRSB or STRSH because STRB/STRH store only the lower bytes and signs don't matter during store.

10.2 Size Variants

```
LDR,  LDRH,  LDRB  ; for 32-bit word, 16-bit halfword, 8-bit byte
STR,  STRH,  STRB  ; for 32-bit word, 16-bit halfword, 8-bit byte
```

11. Addressing Modes in ARM

Addressing modes describe how an instruction specifies the operand. Before the operation is performed, the addressing mode provides a rule for interpreting or modifying the address field of the instruction.

ARM addressing modes fall into two categories:

- For data processing operands: Unmodified value (register/immediate as-is) or Modified value (shifted/rotated)
- For memory access operands: Register indirect, Relative register, Base indexed, Base with scaled register

11.1 Data Processing Addressing Modes

Unaltered value: The register or constant is used as-is, without any shift or rotation.

```
MOV R0, #1234H ; immediate constant 1234 loaded into R0 unchanged
```

Modified value: The register or constant is shifted or rotated. All five shift/rotate operations (LSL, LSR, ASR, ROR, RRX) can be applied.

11.2 Memory Access: Offset Types

Memory is addressed using a base register plus an offset. There are three ways to specify the offset:

Offset Type	Example	Effect
Immediate offset	LDR R0, [R1, #4]	$R0 = \text{mem}[R1 + 4]$
Register offset	LDR R0, [R1, R2]	$R0 = \text{mem}[R1 + R2]$
Scaled register offset	LDR R0, [R1, R2, LSL #2]	$R0 = \text{mem}[R1 + 4 * R2]$

11.3 Pre-index Addressing

The offset is added to the base register to form the address before the memory access. The base register is NOT updated after the access.

```
LDR R0, [R1, #4] ; R0 = mem[R1+4], R1 unchanged
LDR R0, [R1, R2] ; R0 = mem[R1+R2], R1 unchanged
```

11.4 Pre-index with Writeback (Auto-indexing)

Like pre-index, but the effective address (base + offset) is also written back into the base register. Indicated by '!' after the address expression. No extra time penalty.

```
LDR R0, [R1, #4]!    ; R0 = mem[R1+4], then R1 = R1+4
LDR R0, [R1, R2]!    ; R0 = mem[R1+R2], then R1 = R1+R2
```

11.5 Post-index Addressing

The memory access uses the current base register value as the address. The offset is then applied to update the base register after the access. The offset is written OUTSIDE the square brackets.

```
LDR R0, [R1], #4      ; R0 = mem[R1], then R1 = R1+4
LDR R0, [R1], R2      ; R0 = mem[R1], then R1 = R1+R2
```

11.6 Comparison: Pre vs Auto vs Post

Mode	Instruction	R0 =	R1 after =
Pre-indexed	LDR R0, [R1, R2]	mem[R1+R2]	unchanged
Auto-indexing(writ eback)	LDR R0, [R1, R2]!	mem[R1+R2]	R1+R2
Post-indexed	LDR R0, [R1], R2	mem[R1]	R1+R2

11.7 Concrete Example

Given: r0=0x00000000, r1=0x00090000, mem32[0x00009000]=0x01010101, mem32[0x00009004]=0x02020202

Case	Instruction	r0 after	r1 after
Preindex with writeback	LDR r0, [r1, #4]!	0x02020202	0x00009004
Preindex (no writeback)	LDR r0, [r1, #4]	0x02020202	0x00009000 (unchanged)
Postindex	LDR r0, [r1], #4	0x01010101	0x00009004

11.8 Full Addressing Mode Summary

Mode	Syntax	r0 =	r1 +=
Preindex with writeback	LDR r0, [r1, #0x4]!	mem32[r1+0x4]	0x4
	LDR r0, [r1, r2]!	mem32[r1+r2]	r2
	LDR r0, [r1, r2, LSR#0x4]!	mem32[r1+(r2 LSR 0x4)]	(r2 LSR 0x4)

Preindex	LDR r0,[r1,#0x4]	mem32[r1+0x4]	not updated
	LDR r0,[r1,r2]	mem32[r1+r2]	not updated
	LDR r0,[r1,-r2,LSR#0x4]	mem32[r1-(r2 LSR 0x4)]	not updated
Postindex	LDR r0,[r1],#0x4	mem32[r1]	0x4
	LDR r0,[r1],r2	mem32[r1]	r2
	LDR r0,[r1],r2,LSR#0x4	mem32[r1]	(r2 LSR 0x4)